Automated Conference Email Reply & Routing System

Research Design Document — Architecture, Training Strategy & Data Plan

Version 1.0 · Draft — For Review · Pilot Target: AAAI

1. Introduction & Motivation

Major academic conferences — AAAI, NeurIPS, ICLR, ICML — receive between 10,000 and 30,000 submitter inquiries per review cycle. These range from routine FAQ-level questions (deadline clarifications, formatting rules, submission limits) to complex and sensitive cases (reviewer conflicts, dual-submission exceptions, withdrawal requests). Today, every inquiry is handled manually by volunteer Program Chairs — an unsustainable and error-prone process that imposes significant burden during peak periods.

This document defines the research design for an AI-powered system that automates the handling of conference email inquiries. The system's core contribution is a **fully trainable pipeline** in which the classification, retrieval, and routing stages are not hardcoded rules but learned models that improve from historical conference data. A reinforcement learning component enables the routing agent to refine its decisions based on implicit feedback from chairs.

The system is designed from the ground up to operate with **either a locally-hosted model stack or an external API backend**, switchable via a configuration flag. This is a hard constraint: conferences such as AAAI may prohibit transmission of email content to external services, requiring a fully local deployment path.

1.1 Problem Statement

Current conference email management has five distinct failure modes:

Failure Mode	Description	Impact
High redundancy	60-75% of inquiries restate existing FAQ entries	Wasted chair time on zero-value work
Inconsistent answers	Different chairs interpret edge cases differently	Submitters receive contradictory guidance
Routing latency	Messages for specialists sit in general inbox for days	Decisions delayed at critical submission windows
No analytics	No structured record of what submitters struggle with	CFP and website never systematically improved
No reusable infrastructure	Every conference rebuilds ad-hoc process from scratch	Compounding cost across the community

1.2 Research Contributions

This work makes the following contributions:

- **C1** A trainable three-stage pipeline (classify → retrieve → route) for academic conference email, with each stage independently improvable.
- **C2** A reinforcement learning formulation for email routing where chair actions (approve, edit, reroute) serve as the implicit reward signal.
- **C3** A toy benchmark dataset of labelled conference inquiry emails, enabling reproducible evaluation and future comparisons.

C4	A swappable model backend design supporting both local deployment and external API, satisfying institutional data-handling constraints.

2. System Overview

The system intercepts all incoming email to the conference inbox and routes it through a four-stage pipeline. The first three stages — **Classify**, **Retrieve**, and **Route** — are trainable components whose parameters are learned from historical conference data and updated via feedback. The fourth stage — **Draft & Send** — is a generative step that either produces an automated reply (FAQ lane) or a draft for human review (novel lane).

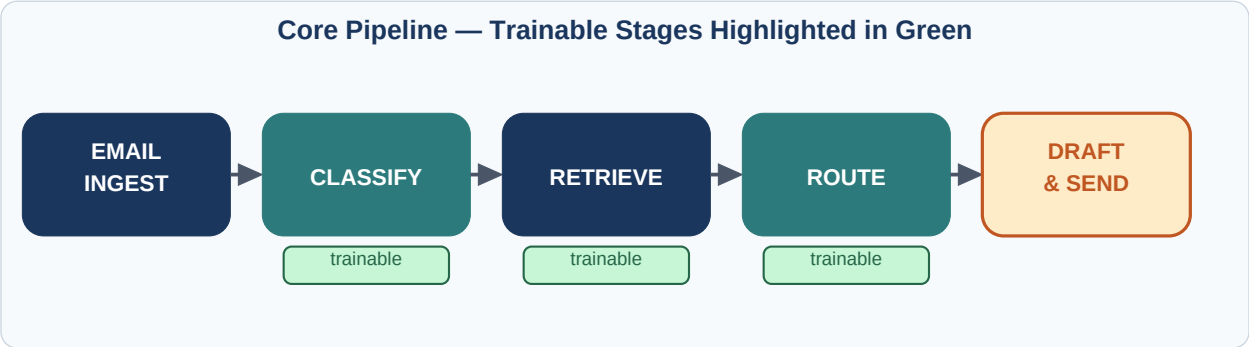

Figure 1 — The four-stage pipeline. Green badges indicate stages that are trained on conference data and updated over time.

2.1 Two-Lane Operating Model

Following the agreed design, the system operates two parallel lanes based on the router's confidence and the classification result:

FAQ lane: The email matches a known policy or FAQ entry with high confidence. A reply is generated from the knowledge base and sent automatically without human involvement. This is appropriate only for questions whose answers are unambiguously stated in the conference's published materials.

Human-review lane: The email is novel, ambiguous, sensitive, or low-confidence. The system generates a draft reply and places it in the chair's approval queue. The chair reviews, optionally edits, and sends. This lane is the default for all non-FAQ traffic and for any email flagged as sensitive by the classifier.

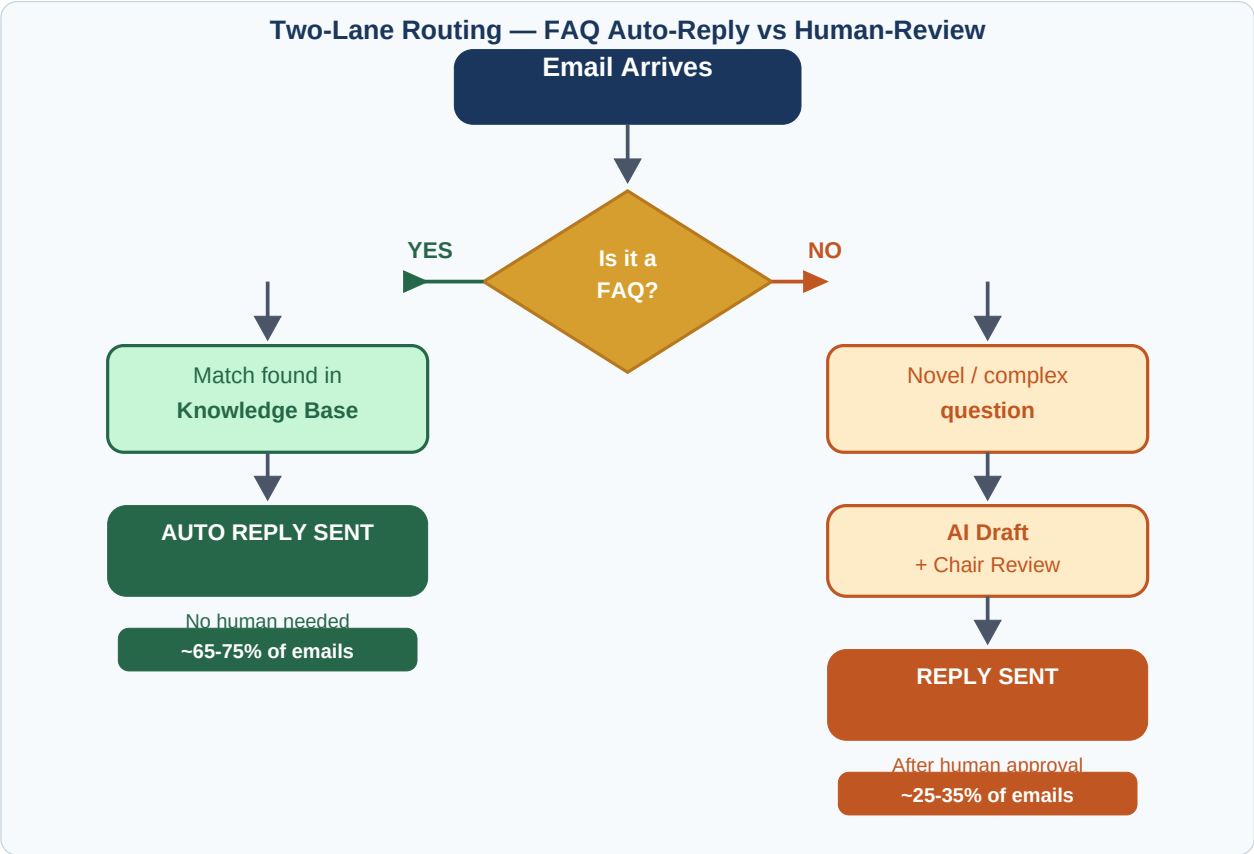

Figure 2 — Two-lane routing. The split point is determined by the router's confidence threshold, configurable per conference.

3. System Architecture

The system is structured as six logical layers. The intelligence layer in the centre contains the three trainable components. The drafting layer below it is backed by either a locally-hosted generative model or an external API, controlled by a single configuration flag.

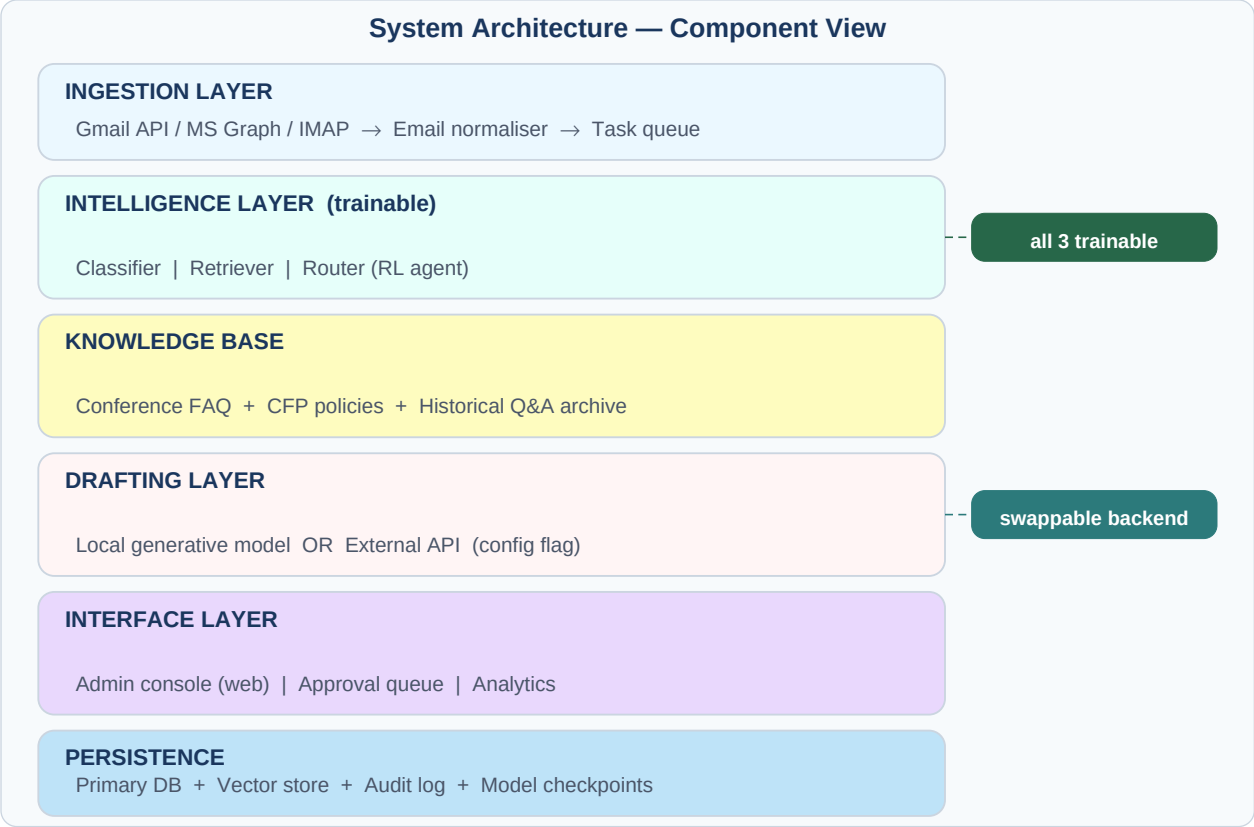

Figure 3 — Component architecture. The intelligence layer (classifier, retriever, router) is fully trainable. The drafting layer backend is swappable.

3.1 Ingestion Layer

The ingestion layer monitors the conference inbox via Gmail API (primary), Microsoft Graph (secondary), or IMAP (fallback). Incoming messages are normalised: quoted reply history is stripped, attachments are catalogued, email threads are reconstructed, and sender metadata is extracted. Normalised messages are pushed to the task queue for async processing.

3.2 Intelligence Layer — Three Trainable Stages

This is the core research contribution. Each of the three sub-components is designed as a learnable module with a clearly defined input/output contract, enabling independent training and evaluation.

3.2.1 Classifier

The classifier takes a normalised email as input and produces a structured output: an **intent label** (e.g. deadline_extension, supplementary_format, withdraw_paper, reviewer_conflict), a set of **extracted entities** (paper IDs, dates, track names), a **confidence score** in [0, 1], and a **sensitivity flag** for emails that should bypass auto-reply regardless of confidence.

The classifier is trained on labelled historical emails using standard text classification techniques. The label schema is defined in collaboration with the conference chairs and is extensible — new intent classes can be added without retraining from scratch. The initial intent taxonomy targets approximately 25-30 labels covering the full range of observed inquiry types.

3.2.2 Retriever

The retriever takes the classified intent and the original email text and returns the most relevant chunks from the knowledge base — the conference FAQ, CFP text, and historical Q&A pairs. Two retrieval strategies are supported:

Phase	Method	When to use
Phase 1 (start)	BM25 keyword search over structured JSON knowledge base	Before real conference data is available. Fast, zero training cost, good baseline for FAQ matching.
Phase 2 (trained)	Dense retrieval with learned query and passage encoders	Once labelled data is available. Handles paraphrase and semantic similarity. Trained using positive/negative pairs from historical replies.

3.2.3 Router

The router is the most novel component. It takes the classifier output (intent, entities, confidence, sensitivity) and the retriever output (top-k knowledge base matches, match scores) and makes a routing decision: which chair role should handle this email, and should it go to the FAQ lane or the human-review lane.

Two routing strategies are designed, deployed progressively:

Strategy	Mechanism	Phase
Rule-based router	Deterministic mapping from intent label to chair role. e.g. reviewer_conflict → Senior PC Chair. FAQ lane if confidence > threshold AND sensitivity = False.	Phase 1 — no training required, immediate deployment
Learned router (RL)	Policy network trained via reinforcement learning. State: classifier + retriever outputs. Action: (lane, chair_role) pair. Reward: implicit feedback from chair actions.	Phase 2 — requires feedback data from live deployment

3.3 Drafting Layer

The drafting layer generates a reply for both lanes. For the FAQ lane, the reply is generated directly from retrieved knowledge base chunks — the generative model is given only the retrieved text and instructed to synthesise a reply without adding information not present in the source. For the human-review lane, the draft is a starting point for the chair to review and edit.

The drafting layer is explicitly **swappable**. A single DRAFTER_BACKEND flag in the configuration file selects between a locally-hosted generative model and an external API. Both satisfy the same interface, so no other code changes when the backend is switched. This directly addresses the AAAI constraint: if external API calls are prohibited, the local backend is activated and no data leaves the institution's infrastructure.

3.4 Interface Layer

The admin console is a browser-based web application. Chairs log in via institutional SSO and see a prioritised queue of drafts awaiting approval. Each queue item shows the original email, the detected intent and confidence, the retrieved source clause, and the AI-generated draft. Three actions are available: **Approve & Send**, **Edit & Send**, and **Reroute**. All actions are logged.

3.5 Persistence

A relational database stores all emails, drafts, sent replies, chair actions, and audit records. The knowledge base is stored in a structured format (JSON in Phase 1, with vector embeddings added in Phase 2 alongside the trained retriever). Model checkpoints are versioned and stored separately to enable rollback.

4. Reinforcement Learning for Routing

The RL routing component is a key research contribution. Rather than requiring manual annotation of routing labels, the system learns optimal routing policy from the implicit feedback that chairs generate naturally as they use the system.

4.1 Problem Formulation

Element	Definition
State (s)	Vector encoding of: intent label, confidence score, sensitivity flag, retriever match scores, email length, sender history (repeat inquirer?), current queue depth per chair role.
Action (a)	A (lane, chair_role) pair. Lane is binary: FAQ or human-review. Chair role is one of: General Inquiries, Publications Chair, Workshop Chair, Registration, Senior PC.
Reward (r)	Derived from chair actions after routing: Approve without edit = +1.0 | Approve with minor edit = +0.5 | Reroute to different chair = -0.5 | Escalate / flag as incorrect = -1.0
Policy (π)	A neural network mapping state → action probability distribution. Trained using a policy gradient algorithm. Updates occur in batches after sufficient feedback accumulates.
Discount (γ)	Routing decisions have near-immediate consequences so γ is set close to 1.0. The key temporal dependency is the chair feedback delay.

4.2 Training Loop

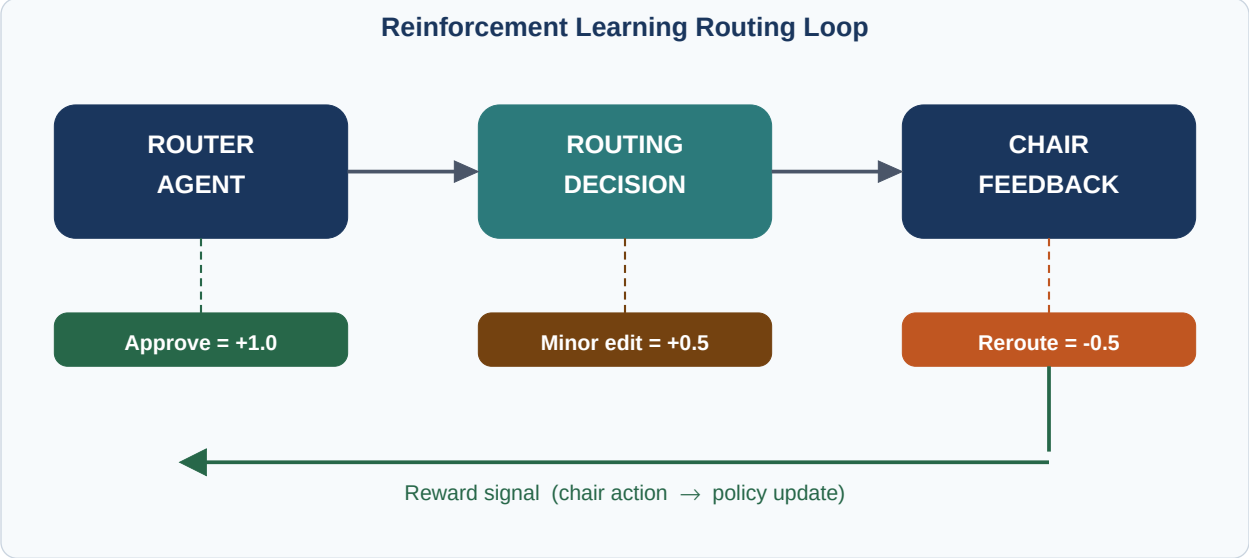

Figure 4 — RL routing loop. Chair actions generate reward signals that update the policy network. No manual routing labels are required.

4.3 Cold-Start Strategy

The RL router cannot be deployed without a warm starting point — a randomly-initialised policy would produce chaotic routing. Two cold-start strategies are planned:

Strategy A — Supervised pre-training: Use the rule-based router to generate pseudo-labels on historical emails. Train the policy network as a supervised classifier on these labels. This gives the RL agent a sensible starting policy before live deployment.

Strategy B — Behaviour cloning from historical data: If the AAAI email archive contains records of who actually handled each email, treat those assignments as expert demonstrations and use imitation learning to initialise the policy. This is the preferred strategy if the data is available and labelled.

4.4 Evaluation of the RL Router

The RL router will be evaluated against the rule-based router baseline on the following metrics: routing accuracy (agreement with expert labels), escalation rate, time-to-resolution per email, and chair satisfaction score. The toy dataset will serve as an offline evaluation benchmark before any live deployment.

5. Training Data Strategy

Three data sources are planned, progressively richer and more realistic. Work begins immediately with the toy dataset while institutional data access is under review.

5.1 Toy Dataset (Immediate)

A manually constructed dataset of 200-300 synthetic emails covering the most common inquiry categories. This serves two purposes: (a) enabling immediate development and iteration on the pipeline before real data arrives, and (b) serving as a public benchmark for future reproducibility if the work is published.

Schema

Field	Type	Description
email_id	string	Unique identifier
email_text	string	Full email body (synthetic, anonymised)
intent_label	enum(~25)	Ground-truth intent class
entities	JSON object	Extracted entities: paper_id, date, track
route_to	enum(5)	Target chair role
lane	binary	0 = FAQ auto-reply, 1 = human review
correct_answer	string	Gold-standard reply text
source_policy	string	CFP / FAQ clause the answer is grounded in
difficulty	enum(3)	easy / medium / hard — for stratified eval

Intent categories to cover (initial taxonomy)

Category	Example intents
Submission mechanics	deadline_extension, format_question, page_limit, supplementary_material, double_submission, withdraw_paper, resubmit_revision
Review process	reviewer_conflict, rebuttal_question, decision_timeline, review_missing, score_clarification
Registration & logistics	registration_fee, visa_letter, presentation_format, camera_ready, copyright_form
Technical platform	openreview_access, submission_portal_error, paper_id_missing
Sensitive / escalate	misconduct_allegation, appeal_request, special_circumstance

5.2 AAAI Historical Email Archive (Pending Approval)

Prof. Yan's access as AAAI Program Chair provides a path to historical conference email data. This dataset, once approved and anonymised, will provide: real intent distribution data, natural language variation in how

submitters phrase questions, historical routing decisions as RL demonstration data, and historical replies as supervised training signal for the drafter. The toy dataset schema is designed to match the expected structure of this real dataset so migration is seamless.

5.3 Feedback Data from Live Deployment

Once the system is deployed in shadow mode (drafts only, never sending automatically), every chair action generates labelled data. Over a single conference cycle, even shadow mode can produce thousands of (state, action, reward) tuples for RL training and classification fine-tuning. This is the long-term training signal that makes the system improve continuously.

6. Literature Review Framework

A literature review is required before finalising the publication strategy. The review will survey four areas and assess the novelty of this work against each.

6.1 Email Routing & Helpdesk Automation

A substantial body of pre-LLM work exists on automated email triage in enterprise and customer support contexts using SVM classifiers, rule-based systems, and early neural approaches. Key papers to survey: routing in contact centres, intent classification for customer service chatbots, and multi-label email categorisation. **Expected gap:** none of this work targets academic conference workflows, none uses RL-based routing, and most predate the era of large pretrained language models.

6.2 Intent Classification in Dialogue Systems

Intent detection is well-studied in task-oriented dialogue (e.g. flight booking, restaurant reservation). Models range from fine-tuned encoders to few-shot prompted LLMs. The conference email setting introduces a domain-specific challenge: intents are tied to a specific policy document (the CFP), so classification and retrieval are tightly coupled in a way not addressed by standard dialogue benchmarks.

6.3 Retrieval-Augmented Generation (RAG)

RAG systems for domain-specific QA are well established. Key references include the original RAG paper (Lewis et al. 2020), subsequent work on dense passage retrieval, and applied RAG systems in legal and medical domains. The conference email case is distinguished by the small, structured nature of the knowledge base and the strict grounding requirement (answers must cite a specific policy clause).

6.4 Reinforcement Learning for Task Routing

RL has been applied to dialogue management (deciding when to hand off to a human agent) and to multi-armed bandit problems in recommendation systems. Direct application to email routing with implicit chair feedback as the reward signal appears to be novel. Key references: RL for dialogue policy learning, deep Q-networks for customer service escalation, and inverse RL for imitation from demonstrations.

6.5 Novelty Assessment

Based on preliminary search, no published system addresses all of: (1) academic conference email specifically, (2) a jointly trainable classify-retrieve-route pipeline, (3) RL-based routing with implicit human feedback, and (4) a swappable local/API backend for institutional compliance. The combination of these four properties is the basis of the publication claim. The literature review will either confirm or refine this assessment.

Area	Key Venues to Search	Expected Finding
Email routing	ACL, EMNLP, SIGIR, CIKM	Pre-LLM methods, no academic conference domain
Intent classification	ACL, NAACL, EMNLP	Strong baselines; our domain is novel
RAG systems	ACL, NeurIPS, ICLR	Well-covered; our grounding constraint is distinctive

Area	Key Venues to Search	Expected Finding
RL for routing	AAAI, NeurIPS, ACL	Dialogue management covered; email routing with implicit rewards is open
Conference tooling	AAAI, NeurIPS workshops	Sparse to nonexistent — strongest novelty signal

7. Implementation Roadmap

Phase	Weeks	Key Deliverables
0 — Foundation	1-2	Repository scaffold with modular architecture. Toy dataset construction begins (200 emails, full schema). Rule-based classifier and router as Phase 1 baseline. BM25 retriever over JSON knowledge base.
1 — Offline Prototype	3-5	End-to-end pipeline running on toy dataset. Classifier trained on toy data — establish intent accuracy baseline. Evaluation framework: accuracy, retrieval MRR, routing precision. Both local and API drafting backends wired up and tested.
2 — Admin Console	5-8	Web-based approval queue for human-review lane. Chair action logging (approve / edit / reroute) — this generates RL data. Knowledge base management UI. Deploy to staging environment.
3 — AAAI Data Integration	8-11	Migrate toy dataset pipeline to AAAI archive (pending approval). Retrain classifier on real data — measure accuracy improvement. Begin dense retriever training with real Q&A; pairs. Shadow mode deployment on AAAI inbox copy.
4 — RL Router	11-16	Implement policy network. Supervised pre-training from rule-based router pseudo-labels. Online RL training from live chair feedback. A/B evaluation: rule-based router vs RL router.
5 — Literature Review & Paper Outline	10-14 (parallel)	Complete literature survey across four areas. Confirm novelty claims. Draft paper outline for target venue. Compile evaluation results for paper.
6 — Pilot & Multi-Tenancy	16-20	Production AAAI pilot (human-in-the-loop). Measure all success metrics vs targets. Multi-tenant refactor for second conference onboarding.

8. Evaluation Metrics

Each trainable component has dedicated evaluation metrics. System-level metrics measure end-to-end performance from the chair's perspective.

Component	Metric	Target
Classifier	Intent accuracy (top-1), F1 per class, confusion matrix	>= 90% accuracy on held-out toy data
Retriever	MRR@5, Recall@10 against gold source clauses	>= 0.75 MRR@5 on annotated pairs
Rule-based router	Routing accuracy vs expert labels	>= 85% agreement
RL router	Cumulative reward, escalation rate, routing accuracy	Outperforms rule-based on held-out feedback
Drafter	Draft acceptance rate (no edits), BERTScore vs gold replies	>= 65% acceptance; BERTScore F1 >= 0.80
System — speed	Median time from email receipt to reply sent	< 5 minutes

Component	Metric	Target
System — deflection	% of emails resolved without any chair intervention	>= 50% in pilot
System — cost	Cost per resolved inquiry (infra + model API if used)	< $0.05 per email

9. Swappable Backend Design

The swappable backend is a first-class architectural requirement, not an afterthought. Every model-backed component implements a shared abstract interface. A single configuration file controls which implementation is active. No application code changes when the backend is switched.

Component	Local Backend	API Backend	Switch
Classifier	Fine-tuned encoder model (runs on CPU)	Prompted LLM via API	CLASSIFIER_BACKEND
Retriever	BM25 (Phase 1) / Dense retriever (Phase 2)	Hosted embedding API + vector search	RETRIEVER_BACKEND
Router	Local policy network (rule-based or RL)	N/A — router always runs locally	ROUTER_MODE
Drafter	Locally-hosted generative model via Ollama	External LLM API	DRAFTER_BACKEND

The local backend is the primary deployment target. AAAI and similar conferences may prohibit external API calls due to data privacy and confidentiality obligations. All components in the local path must run on standard academic computing hardware without requiring GPU clusters. The API backend is provided as a development convenience and for conferences without local deployment constraints.